

# Homework 3

羊瑞

2300013091

2026 年 4 月 5 日

## Problem 1

4.3 设有一条边远山区的道路  $AB$ , 沿着道路  $AB$  分布着  $n$  所房子, 这些房子到  $A$  的距离分别是  $d_1, d_2, \dots, d_n$  ( $d_1 < d_2 < \dots < d_n$ ). 为了给所有房子的用户提供移动电话服务, 需要在这条道路上设置一些基站. 为了保证通信质量, 每所房子应该位于距离某个基站的 4 千米范围之内. 设计一个算法找到基站的位置, 并且使得基站总数达到最少. 用文字说明算法的主要设计思想; 给出算法的伪码描述; 证明算法的正确性并给出算法最坏情况下的时间复杂度函数.

### Solution.

若基站设在位置  $x$ , 则它能覆盖区间  $[x - 4, x + 4]$ .

贪心思想: 每次先考虑当前最左边尚未被覆盖的房子, 记其位置为  $d_i$ . 为了让这个基站尽可能多地覆盖后面的房子, 应当把基站放在该房子能够被覆盖的最靠右位置, 即  $x = d_i + 4$ . 这样它能覆盖区间  $[d_i, d_i + 8]$ , 从而尽可能多地覆盖后续房子. 然后跳过所有已被该基站覆盖的房子, 继续处理下一个未覆盖房子.

### Algorithm:

```
1: Input:  $d_1 < d_2 < \dots < d_n$ 
2: Output: 基站位置集合  $B$ 
3:  $B \leftarrow \emptyset, i \leftarrow 1$ 
4: while  $i \leq n$  do
5:    $x \leftarrow d_i + 4$ 
6:    $B \leftarrow B \cup \{x\}$ 
7:   while  $i \leq n$  and  $d_i \leq x + 4$  do
8:      $i \leftarrow i + 1$ 
9:   end while
10: end while
11: Output  $B$ 
```

正确性说明: 设当前最左边未覆盖的房子为  $d_i$ . 任何可行解都必须用某个基站覆盖它, 而该基站位置不能超过  $d_i + 4$ , 否则  $d_i$  将得不到覆盖. 因此, 把该基站放在  $d_i + 4$  是所有可行选择中最靠右的

位置, 这样不会减少对左端房子的覆盖, 同时能使对右侧房子的覆盖范围最大, 从而不会比任何其他放法更差。于是, 贪心地在  $d_i + 4$  处设基站后, 问题就转化为对剩余未覆盖房子的同类子问题。故该贪心策略是最优的。

时间复杂度为  $O(n)$ 。

□

## Problem 2

4.6 有  $n$  项作业的集合  $J = \{1, 2, \dots, n\}$ , 每项作业  $i$  有加工时间  $t(i) \in \mathbf{Z}^+$ . 有一台机器从时刻 0 开始工作, 直到完成所有的任务. 一个可行调度  $f$  是对  $J$  中任务的一个安排, 对于  $i \in J$ ,  $f(i)$  是任务  $i$  开始加工的时间,  $f$  满足下述条件:

$$f(i) + t(i) \leq f(j) \quad \text{或} \quad f(j) + t(j) \leq f(i), \quad j \neq i, \quad i, j \in J$$

设作业  $i$  的完成时间  $w(i) = f(i) + t(i)$ , 求使得平均完成时间  $\frac{1}{n} \sum_{i=1}^n w(i)$  最少的调度.

### Solution.

设作业  $i$  的加工时间为  $t(i)$ , 完成时间为  $w(i) = f(i) + t(i)$ . 目标是最小化平均完成时间  $\frac{1}{n} \sum_{i=1}^n w(i)$ , 而  $n$  是常数, 等价于最小化  $\sum_{i=1}^n w(i)$ .

贪心: 应当优先安排加工时间短的作业, 即按加工时间从小到大排序后依次执行.

### Algorithm:

- 1: Input: 作业集合  $J = \{1, 2, \dots, n\}$ , 加工时间  $t(i)$
- 2: Output: 一个最优调度
- 3: 按  $t(i)$  从小到大对所有作业排序
- 4: 按排序后的顺序依次加工各作业

正确性说明: 设某个调度中有相邻两个作业  $i, j$ , 并且  $t(i) > t(j)$ , 但调度顺序却是  $i$  在前、 $j$  在后。设它们之前所有作业的总加工时间为  $T$ 。

则按顺序  $i, j$  排列时, 这两个作业对总完成时间的贡献为  $(T + t(i)) + (T + t(i) + t(j)) = 2T + 2t(i) + t(j)$ 。

若交换顺序为  $j, i$ , 贡献变为  $(T + t(j)) + (T + t(j) + t(i)) = 2T + 2t(j) + t(i)$ 。

两者之差为  $(2T + 2t(i) + t(j)) - (2T + 2t(j) + t(i)) = t(i) - t(j) > 0$ 。

因此交换后总完成时间变小。所以, 在最优调度中, 不可能存在“较长作业排在较短作业前面”的逆序对。于是最优调度一定是按加工时间非降序排列的。

时间复杂度:  $O(n \log n)$ 。

□

## Problem 3

4.13 设有作业集合  $J = \{1, 2, \dots, n\}$ , 每项作业的加工时间都是 1. 所有作业的截止时间是  $D$ . 若作业  $i$  在  $D$  之后完成, 则称为被延误的作业, 并需要赔偿罚款  $m(i)$ . 这里的  $D$  和  $m(i) (i=1, 2, \dots, n)$  都是正整数, 且  $n$  项  $m(i)$  彼此不等.  $J$  的一个调度是函数  $f: J \rightarrow \mathbb{N}$ , 其中  $\mathbb{N}$  为自然数集合,  $f(i)$  表示作业  $i$  开始加工的时间,  $i=1, 2, \dots, n$ . 设计一个算法求出使得总罚款最少的调度, 证明算法的正确性并给出最坏情况下的时间复杂度.

### Solution.

由于每个作业长度都相同, 若想使总罚款最小, 就要在  $D$  个时段内尽量安排罚款最大的  $D$  个作业按时完成, 其余作业延迟完成.

贪心: 按罚款  $m(i)$  从大到小排序, 选出前  $\min\{D, n\}$  个作业作为“按时完成”的作业集合, 其余作业作为延迟作业. 因为所有按时作业长度都为 1, 所以只要按任意顺序把这至多  $D$  个作业安排在时刻  $0, 1, \dots, D-1$  上即可, 它们都能在截止时间前完成.

### Algorithm:

- 1: Input: 作业集合  $J = \{1, 2, \dots, n\}$ , 共同截止时间  $D$ , 罚款  $m(i)$
- 2: Output: 使总罚款最小的调度
- 3: 按  $m(i)$  从大到小对作业排序
- 4: 取前  $\min\{D, n\}$  个作业组成集合  $A$
- 5: 其余作业组成集合  $B$
- 6: 将  $A$  中作业安排在时刻  $0, 1, \dots, |A| - 1$
- 7: 将  $B$  中作业安排在时刻  $|A|, |A| + 1, \dots, n - 1$

正确性说明: 由于每个作业加工时间都为 1, 最多只有  $D$  个作业能够在截止时间  $D$  前完成. 设某个最优解中, 按时完成的作业集合为  $A$ , 延迟作业集合为  $B$ . 如果存在  $i \in A, j \in B, m(j) > m(i)$ , 则交换  $i, j$  的角色: 令  $j$  按时完成、 $i$  延迟完成. 交换后总罚款减少了  $m(j) - m(i) > 0$ , 这与原解最优矛盾. 因此, 在任一最优解中, 按时完成的必然是罚款最大的那些作业. 于是选择罚款最大的前  $D$  个作业按时完成是最优的.

时间复杂度:  $O(n \log n)$ .

□

## Problem 4

4.18 设  $S = \{1, 2, \dots, n\}$  是  $n$  项广告的集合, 广告  $i (i=1, 2, \dots, n)$  有发布开始时间  $s(i)$  和截止时间  $d(i)$ , 发布效益是  $v(i)$ , 其中  $s(i)$  是非负整数,  $d(i)$  和  $v(i)$  是正整数, 且  $d(1) \leq d(2) \leq \dots \leq d(n)$ . 我们的问题是: 如何在  $S$  中选择一组广告  $A$ , 使得  $A$  中任意两个广告都相容 (时间段不重叠) 且总效益最大?

—— (1) 假设所有广告的效益都相等, 试设计一个求解上述问题的算法, 证明其正确性, 并说明时间复杂度.

—— (2) 如果效益  $v(i)$  可以取任意正整数, 设计一个算法求解这个问题, 用文字说明算法的设计思想和主要步骤, 分析算法最坏情况下的时间复杂度.

### Solution.

每个广告  $i$  有开始时间  $s(i)$ 、结束时间  $d(i)$  和效益  $v(i)$ ，并且已经满足  $d(1) \leq d(2) \leq \dots \leq d(n)$ 。  
 (1) 当所有广告效益都相等时选择尽可能多的互相兼容的广告。最优策略是：每次选择当前结束时间最早且与已选广告兼容的广告。

**Algorithm:**

- 1: 按结束时间从小到大扫描广告
- 2: 先选第一个广告
- 3: 之后每次选取开始时间不早于上一个已选广告结束时间的第一个广告

正确性说明：结束时间最早的广告为后续广告留下的可用时间最多，因此不会比选择其他广告更差。利用交换论证可证明该贪心算法最优。

时间复杂度： $O(n)$ 。

(2) 当效益  $v(i)$  可为任意正整数时先定义  $p(i) = \max\{j < i \mid d(j) \leq s(i)\}$ ，即与广告  $i$  兼容的、结束时间最晚的广告下标；若不存在，则令  $p(i) = 0$ 。

再定义状态  $dp[i]$  表示只考虑前  $i$  个广告时可获得的最大总效益。

则递推关系为  $dp[i] = \max\{dp[i-1], dp[p(i)] + v(i)\}$ 。

- 不选广告  $i$ ，则最优值为  $dp[i-1]$ ；
- 选广告  $i$ ，则其之前只能接一个与之兼容的最优方案，即  $dp[p(i)]$ ，总效益为  $dp[p(i)] + v(i)$ 。

初始条件： $dp[0] = 0$

最终答案为  $dp[n]$

**Algorithm:**

- 1: 对每个  $i$  计算  $p(i)$
- 2:  $dp[0] \leftarrow 0$
- 3: **for**  $i = 1$  to  $n$  **do**
- 4:      $dp[i] \leftarrow \max\{dp[i-1], dp[p(i)] + v(i)\}$
- 5: **end for**
- 6: 输出  $dp[n]$

正确性说明：对前  $i$  个广告的最优方案，最后一个广告只有两种可能：选  $i$  或不选  $i$ 。若不选，则问题退化为前  $i-1$  个广告的最优解；若选，则前面只能接到  $p(i)$  为止的最优解。因此状态转移完整且正确。

时间复杂度：对每个  $i$  用二分查找求  $p(i)$ ，复杂度为  $O(n \log n)$ ，动态规划部分为  $O(n)$ 。 □

## Problem 5

5.2 最小重量机器设计问题：某设备需要 4 种配件，每种 1 件。有 3 个供应商提供这些配件，表 5.2 给出相关的价格和每种配件的重量；从中选择这 4 种配件，使得总价值不超过 120，且总重量最轻。

表 5.2 产品和供应商信息表

| 零件编号 | 供应商 1 |    | 供应商 2 |    | 供应商 3 |    |
|------|-------|----|-------|----|-------|----|
|      | 价格    | 重量 | 价格    | 重量 | 价格    | 重量 |
| 1    | 10    | 5  | 8     | 6  | 12    | 4  |
| 2    | 20    | 8  | 21    | 10 | 30    | 5  |
| 3    | 40    | 5  | 42    | 4  | 30    | 10 |
| 4    | 30    | 20 | 60    | 10 | 45    | 15 |

### Solution.

设第  $i$  种配件从第  $j$  个供应商购买时，价格为  $p_{ij}$ ，重量为  $w_{ij}$ 。则目标是最小化  $\sum_{i=1}^4 w_{i,x_i}$  并满足  $\sum_{i=1}^4 p_{i,x_i} \leq 120$ ，其中  $x_i \in \{1, 2, 3\}$ 。

由于规模很小，可以直接枚举所有方案，共有 81 种。逐一计算总价格和总重量，在满足预算约束的前提下取总重量最小者。

根据表中数据，经枚举可得最优方案为：

- 配件 1：选供应商 3，价格 12，重量 4；
- 配件 2：选供应商 1，价格 20，重量 8；
- 配件 3：选供应商 2，价格 42，重量 4；
- 配件 4：选供应商 3，价格 45，重量 15。

于是总价格为  $12 + 20 + 42 + 45 = 119 \leq 120$ ，总重量为

$$4 + 8 + 4 + 15 = 31.$$

因此最优解为：选择 (3,1,2,3) 号供应商，最小总重量为 31。

时间复杂度：枚举总数为  $3^4 = 81$ ，因此时间复杂度为  $O(3^4)$ 。在一般情况下，若有  $n$  类配件、每类有  $m$  个供应商，则复杂度为  $O(m^n)$ 。

□

## Problem 6

5.5 给出 8 皇后问题的一个广度优先回溯算法，并分析该算法的时间复杂度。

### Solution.

按行逐层扩展状态，第  $k$  层表示前  $k$  行已经放好了皇后。状态表示：用一个长度为  $k$  的向量  $(x_1, x_2, \dots, x_k)$  表示前  $k$  行的放置情况，其中  $x_i$  表示第  $i$  行皇后所在列。

结点扩展规则：从第  $k$  层状态扩展到第  $k+1$  层时，尝试把第  $k+1$  行皇后放在每一列  $j \in \{1, \dots, 8\}$ ，只保留满足以下条件的位置：

$$x_i \neq j, \quad |x_i - j| \neq |i - (k+1)|, \quad i = 1, 2, \dots, k.$$

广度优先算法：

```

1: 将空状态入队
2: while 队列非空 do
3:   取出队首状态  $(x_1, \dots, x_k)$ 
4:   if  $k = 8$  then
5:     输出一个解
6:   else
7:     for  $j = 1$  to 8 do
8:       if 将第  $k+1$  行皇后放在第  $j$  列合法 then
9:         将新状态  $(x_1, \dots, x_k, j)$  入队
10:      end if
11:    end for
12:  end if
13: end while

```

正确性说明：该算法按层枚举所有可能的部分解，并在每一步利用约束条件剪去不可能扩展成解的状态，因此不会漏掉任何合法解；当扩展到第 8 层时得到的恰好是完整解。

时间复杂度：最坏情况下，每一层最多有 8 个分支，因此状态空间上界为  $O(8^8)$ 。考虑到同列冲突的限制，更精确的上界可写成  $O(8!)$ 。

□

## Problem 7

5.6 子集和问题. 设  $n$  个不同的正数构成集合  $S$ , 求出使得和为某数  $M$  的  $S$  的所有子集

### Solution.

状态表示：设当前处理到第  $k$  个元素，当前已选元素和为  $sum$ 。

- 若  $sum = M$ ，则得到一个解；
- 若  $sum > M$ ，由于所有数均为正整数，不可能再得到解，立即回溯；
- 若  $k > n$  仍未达到  $M$ ，则回溯。

回溯算法：

```

1: procedure BACKTRACK( $k, sum$ )

```

```

2:   if  $sum = M$  then
3:       输出当前子集
4:       return
5:   end if
6:   if  $k > n$  or  $sum > M$  then
7:       return
8:   end if
9:   选择  $a_k$ 
10:  BACKTRACK( $k + 1, sum + a_k$ )
11:  不选择  $a_k$ 
12:  BACKTRACK( $k + 1, sum$ )
13: end procedure

```

初始调用为 Backtrack(1, 0).

正确性说明：每个元素都被完整地考虑了“选”与“不选”两种情况，因此算法遍历了所有可能子集。由于剪枝仅发生在当前和已超过  $M$  时，而所有元素都为正，因此这些分支不可能再形成合法解，故剪枝是安全的。

时间复杂度： $O(2^n)$

□

## Problem 8

5.7 分派问题. 给  $n$  个人分配  $n$  件工作, 给第  $i$  个人分配第  $j$  件工作的成本是  $C(i, j)$ . 试求成本最小的工作分配方案.

### Solution.

状态表示：按人依次分配工作。设当前已经给前  $k - 1$  个人分配了工作，当前成本为  $cost$ ，并记录哪些工作已被使用。给第  $k$  个人尝试分配一个尚未被使用的工作  $j$ ，然后递归处理第  $k + 1$  个人。当  $k = n + 1$  时，说明所有人都完成分配，此时若总成本更小，就更新最优解。

Algorithm:

```

1:  $best \leftarrow +\infty$ 
2: procedure BACKTRACK( $k, cost$ )
3:   if  $k = n + 1$  then
4:       if  $cost < best$  then
5:            $best \leftarrow cost$ 
6:           记录当前分配方案
7:       end if
8:       return
9:   end if

```

```
10:   for 每个尚未使用的工作  $j$  do
11:       if  $cost + C(k, j) < best$  then
12:           将工作  $j$  分给第  $k$  个人
13:           标记  $j$  已使用
14:           BACKTRACK( $k + 1, cost + C(k, j)$ )
15:           取消标记
16:       end if
17:   end for
18: end procedure
```

初始调用为 Backtrack(1, 0).

正确性说明：该算法枚举了所有可能的——分配方案。剪枝条件  $cost + C(k, j) \geq best$  表示即使继续向下搜索，当前部分解的总代价也不会优于已知最优解，因此可以安全剪去，不影响最优性。

时间复杂度： $O(n!)$

□